

Name : Rafael

Student ID : 32215130

Major : Global Software Convergence

Freedays : 0

Introduction

In modern operating systems, effective CPU scheduling is essential for efficient multitasking and resource utilization. This project simulates a basic round-robin scheduling algorithm integrated with I/O burst handling using message queues for inter-process communication (IPC). It demonstrates the intricacies of process management, queuing mechanisms, and real-time scheduling in a controlled environment.

Objective

The primary goal of this project is to implement a simplified CPU scheduling mechanism using:

1. Round-robin scheduling with a fixed time quantum.
 2. Handling of I/O bursts where processes temporarily leave the CPU queue for I/O operations.
 3. Real-time logging to monitor the state of processes in both run and wait queues.
 4. IPC mechanisms to simulate communication between processes and the scheduler.
-

Implementation Details

Key Components

1. **Round-Robin Scheduler**
 - Fixed time quantum (`TIME_QUANTUM`) is set to simulate time-slicing for processes.
 - The scheduler maintains two queues: `run_queue` and `wait_queue` to handle CPU bursts and I/O bursts, respectively.
2. **Inter-Process Communication (IPC)**
 - A System V message queue facilitates communication between the scheduler and child processes.

- Each process sends its I/O burst information to the scheduler via messages.
- 3. **Process Simulation**
 - A parent process forks multiple child processes, each simulating independent workloads with CPU and I/O bursts.
 - Child processes periodically send I/O requests and simulate I/O operations.
- 4. **Logging and Monitoring**
 - A log file (`schedule_dump.txt`) records the state of processes, queue transitions, and scheduling decisions.

Compilation and Execution

Compile the parent and child process code:

```
gcc parent.c -o parent
```

1. `gcc child.c -o child`
2. Run the parent:

```
./parent
```

Ensure the `child` binary is in the same directory, as the parent invokes it via `exec1`.

Challenges and Learning Experiences

1. **Queue Management**
 - Ensuring accurate transitions of processes between `run_queue` and `wait_queue` required careful handling of pointers and memory management.
 - Implementing the removal of completed I/O burst processes from the wait queue was particularly challenging.
2. **IPC Synchronization**
 - Handling race conditions in message queue operations involved meticulous use of non-blocking flags (`IPC_NOWAIT`).
3. **Signal Handling**
 - Coordinating between signals (`SIGALRM`) and queue updates demanded a clear understanding of asynchronous execution.
4. **Debugging and Testing**
 - Debugging inter-process communication and observing queue states required thorough logging and iterative testing.

Unique Features of the Code

1. **Dynamic Workload Management**
 - CPU bursts are dynamically adjusted for processes returning from I/O, simulating real-world workload variations.
2. **Time-Driven Simulation**
 - The use of periodic signals (`SIGALRM`) creates a time-driven simulation of process scheduling.
3. **Comprehensive Logging**

- The log file provides a clear and detailed trace of all scheduling decisions and queue states, aiding in analysis and debugging.
-

Possible Future Enhancements

1. **Priority Scheduling**
 - Incorporate process priority levels to simulate more complex scheduling algorithms.
 2. **Graphical Visualization**
 - Develop a visualization tool to display queue transitions and CPU usage dynamically.
 3. **Multi-Core Simulation**
 - Extend the scheduler to support multi-core CPU simulation, where processes can be assigned to different cores.
 4. **Advanced I/O Modeling**
 - Introduce varied I/O operations with different device characteristics and service times.
-

ChatGPT Usage

In this project, I utilized ChatGPT for assistance in understanding key concepts and debugging specific portions of the code. Below are instances where ChatGPT's help was particularly valuable:

1. **Thread Synchronization**
 - **Challenge:** Implementing proper synchronization between processes using message queues and ensuring safe transitions between run and wait queues.
 - **ChatGPT Assistance:** ChatGPT explained how to use message queues effectively in a producer-consumer model and clarified the usage of `IPC_NOWAIT` flags for non-blocking communication.
2. **Signal Handling**
 - **Challenge:** Coordinating `SIGALRM` signals with queue updates while avoiding race conditions.
 - **ChatGPT Assistance:** Provided clear examples of using `sigaction` and `setitimer` to handle periodic signals safely in a multitasking environment.
3. **Queue Operations**
 - **Snippet:** Queue Initialization and Enqueue Operation

```

void queueINIT(Queue* q) {
    q->front = q->back = NULL;
}

int empty(Queue* q) {
    return q->front == NULL;
}

void enqueue(Queue* q, int pid, int remaining_cpu_burst, int io_burst) {
    Node* temp = (Node*)malloc(sizeof(Node));
    if (!temp) {
        perror("malloc failed");
        exit(1);
    }
    temp->pid = pid;
    temp->CPUBurstRemain = remaining_cpu_burst;
    temp->burstIO = io_burst;
    temp->next = NULL;
    if (q->back) {
        q->back->next = temp;
    }
    q->back = temp;
    if (!q->front) {
        q->front = temp;
    }
}

```

ChatGPT helped clarify how to structure this function to handle both empty and non-empty queue cases properly.

4. **Process Re-Enqueue After I/O**
 - **Snippet:** Handling I/O Completion

```

Node* prev = NULL, * curr = queueWAIT.front;
while (curr) {
    curr->burstIO--;
    if (curr->burstIO <= 0) {
        enqueue(&queueRUN, curr->pid, rand() % MAX_BURST + 1, 0);
        fprintf(fileLOG, "Process %d finished I/O and moved to run queue\n", curr->pid);
        fflush(fileLOG);

        if (prev) {
            prev->next = curr->next;
        }
        else {
            queueWAIT.front = curr->next;
        }
        Node* temp = curr;
        curr = curr->next;
        free(temp);
        continue;
    }
    prev = curr;
    curr = curr->next;
}

```

○

ChatGPT provided guidance on how to update the wait queue dynamically and transition processes back to the run queue upon I/O completion.

5. Child Process Simulation

- **Snippet:** Child Process I/O Handling
-

ChatGPT explained how to simulate the child process's behavior for sending I/O requests and resetting CPU bursts after I/O completion.

Conclusion

This project demonstrates the implementation of a round-robin scheduling algorithm with I/O handling. By leveraging message queues and periodic signals, it provides a simplified yet realistic simulation of process management in an operating system. Through this exercise, we gained valuable insights into CPU scheduling, IPC mechanisms, and real-time process monitoring. Future enhancements can build upon this foundation to explore more complex and advanced scheduling scenarios.